

PACE

Predictive Adaptive Context Extraction for Long-Horizon LLM Agents - 深度研究与工程 Know-how

把 Context Management 重写成 Next-Step Prediction：不是“压缩全部历史”，而是“下一步应该读哪些历史、读到多细”

长程 Agent Context Management：从“压缩历史”到“预测下一步需要的历史”

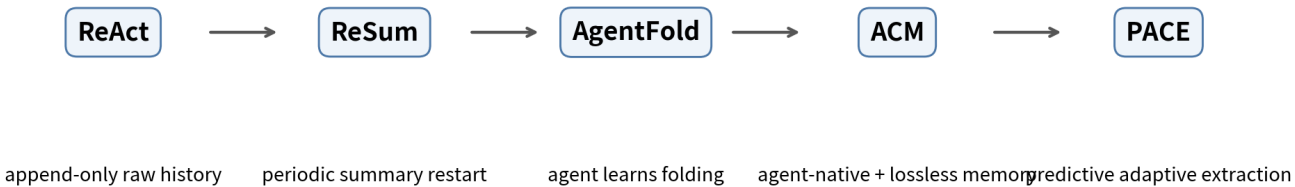


图 1 PACE 在 Context Engineering 演进中的位置

研究日期：2026-08-13 | 主来源：Wei et al., ACL 2026 (2026.acl-long.1252)

目录与执行摘要

1. 一页结论：PACE 真正解决的是什么
2. Know-why：为什么“保存全部历史”和“不断总结历史”都会失败
3. PACE 架构：External Memory 与 Dynamic Context View
4. Multi-Granularity Memory：四层历史表示
5. Predictive Attention：把历史检索改写成 Next-Step Prediction
6. Pressure-Adaptive Compression：预算变成反馈控制信号
7. Glimpse + Async Summaries：为什么 PACE 可以近乎零额外延迟
8. 实验体系与主结果
9. Ablation：真正重要的组件是什么
10. Context Growth：从线性膨胀到有界波动
11. Task Complexity：任务越难，PACE 越有价值
12. Ultra-Long Horizon：4,897 steps 如何得到
13. Hyperparameter 与 Runtime Know-how
14. PACE vs ReSum / AgentFold / LongSeeker / AdaCoM / ACM
15. 生产级 Deep Research 如何复用 PACE
16. 局限、风险与下一步演进
17. 结论与资料来源

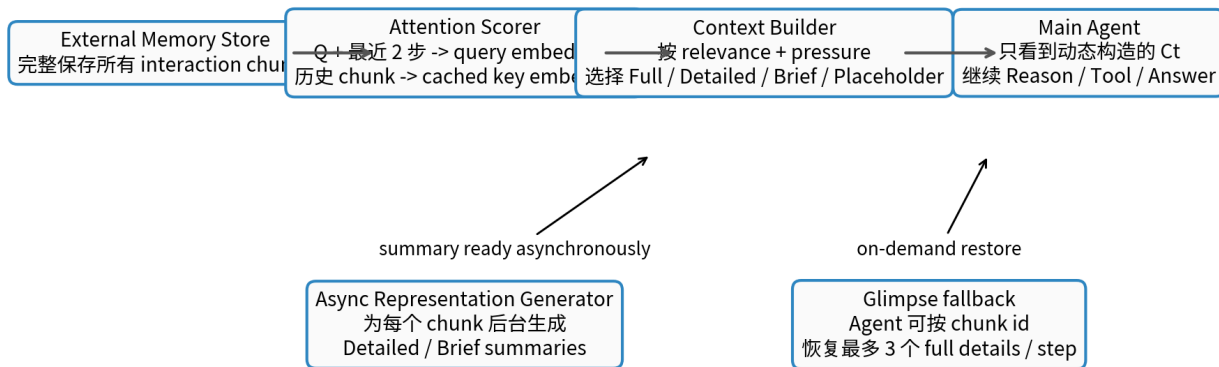
执行摘要：PACE 的核心不是训练一个更强的“摘要 Agent”，而是把完整 trajectory 放在 External Memory Store 中，每个 chunk 建立多粒度表示与 cached embedding。每一步，系统用“原始问题 + 最近两步完整交互”形成 query，预测每个旧 chunk 对下一步 action 的相关性，再根据 relevance 和 compression pressure 自动选择 Full / Detailed / Brief / Placeholder。主 Agent 不需要学习新的 memory skill；Context Builder 变成一个独立、training-free、可插拔的 read-path。

1. 一页结论：PACE 真正解决的是什么

PACE 研究的是一个比“如何摘要历史”更基础的问题：对长程 Agent 来说，每一步真正需要的不是“过去发生过什么的统一摘要”，而是“为了下一步决策，现在最值得重新看到哪些历史”。这使 Context Management 从 History Compression 转为 Predictive Context Extraction。

- External Memory Store 永久保留完整 interaction history；active context 只是一个动态 view。
- 每个历史 chunk 有四层表示：Full、Detailed Summary、Brief Summary、Placeholder。
- Attention Scorer 使用 BGE-M3，把 Q + 最近 N=2 个 full chunks 编码成 q_t ；旧 chunk 的 key embedding 只计算一次并缓存。
- 低温 softmax $\tau=0.3$ 把 relevance 分布拉尖；relative weight 决定历史 chunk 的表示粒度。
- Compression Pressure $P_t = \max(t/T_{\max}, |C_{t-1}|/B_{\max})$ ；随着任务变长或 context 逼近预算，阈值自动上升。
- Glimpse 是 fallback：如果某个历史块被压得太狠，Agent 可按 chunk id 恢复 full content；每一步最多 3 个。
- 论文主结果：四种模型、六项 benchmark 中 PACE 全面优于 ReAct / Summary / Folding；在 Tongyi DeepResearch 30B 上 xbench-DR 达 81.0。
- Stress Test：256K context budget 下，ReAct 74 steps、Summary 233、Folding 954、PACE $\lambda=.5$ 为 2,776、 $\lambda=1.0$ 为 4,897。

PACE：External Memory + Predictive Attention + Multi-Granularity Context Builder



关键：PACE 不修改主 Agent 权重，也不要求主 Agent 学习额外的“fold”元认知技能

图2 PACE 不是新的 Agent Policy，而是新的 Context Read Path

Source: Wei et al., “PACE: Predictive Adaptive Context Extraction for Long-Horizon LLM Agents”, ACL 2026, Sec. 3-4.

2. Know-why：为什么两种经典方案都会失败

2.1 Full-history ReAct：完整 ≠ 有效

传统 ReAct 的直觉是：只要窗口够大，就保留全部 history。PACE 指出这同时带来两个问题：第一是 token/attention 成本不断增长；第二是 information density 持续下降。大量低价值、失败分支、重复 observation 会稀释真正与下一步相关的证据。于是问题不仅是“窗口超限”，也是“上下文内部信噪比下降”。

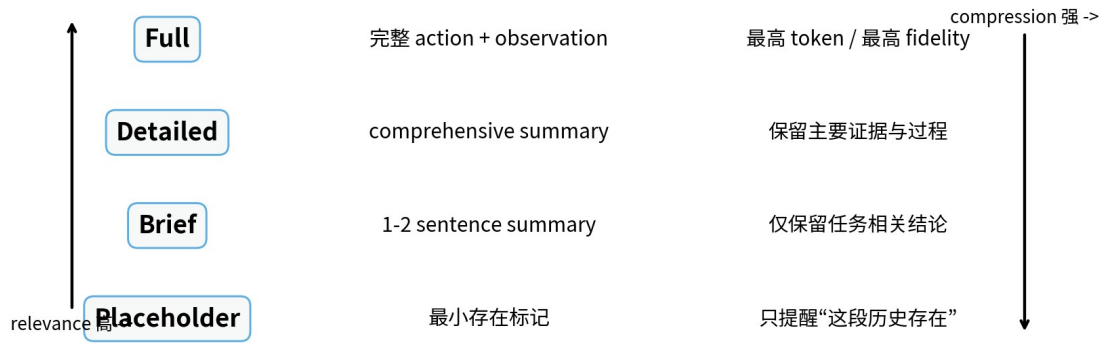
2.2 Summary Agent：压缩 ≠ 选择

累计摘要能缓解 token 增长，但通常用一个统一 granularity 重写历史。它不知道下一步要做什么，因此无法提前知道哪些细节应该保留。一个当时看似无关的 URL、数值、实体或约束，可能在几十步后成为关键。全局 summary 的不可逆丢失会把这种信息永久擦除。

2.3 Folding Agent：更聪明，但把 memory skill 绑进主 Agent

AgentFold/Context-Folding 让主 Agent 生成 fold 指令，已经从静态压缩走向主动 context management。但 PACE 的批评是：这把“完成任务”与“学会怎样整理记忆”耦合进同一个 policy。对于未经过对应 fine-tuning 的通用模型，这种 meta-cognitive skill 不一定稳定。PACE 因此选择外置一个 predictive Context Builder。

同一个历史 Chunk 的四层表示：信息不是“留/删”二元选择



PACE 的核心 know-how：不要只训练“更好的 summary”；先设计多分辨率 memory representation，再让 selection 决定读哪一层。

图3 PACE 先把 Memory 设计成多分辨率，再讨论“选择什么”

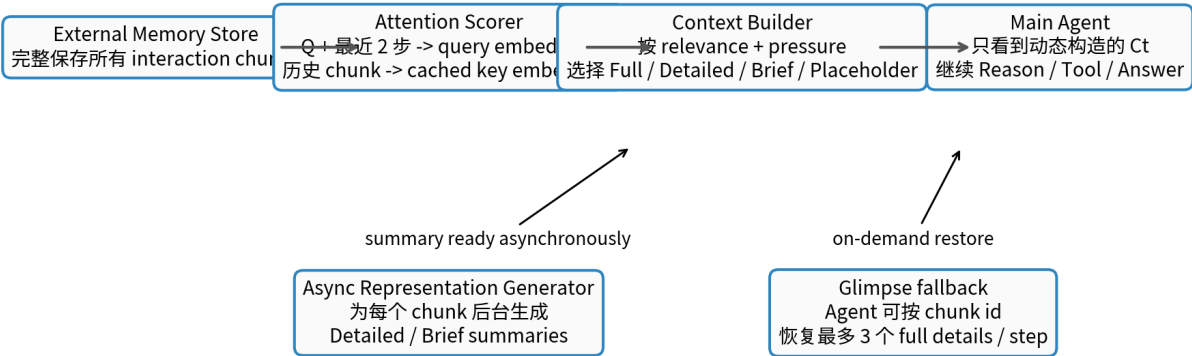
3. PACE 架构：External Memory 与 Dynamic Context View

论文把系统拆成五个协调组件：Main Agent、External Memory Store、Representation Generator、Attention Scorer、Context Builder。重要的是把“系统真实状态”和“模型本轮看到的上下文”彻底分开。

组件	状态/职责	工程含义
Main Agent	接收 C _t , 执行 reasoning / tool action	可以替换成 WebSailor、Tongyi DR、DeepSeek、Claude 等，不必训练 PACE-specific skill
External Memory Store M	完整历史 Chunk ₁ ...Chunk _{t-1}	Active context 可丢细节，但底层 evidence/trace 不必丢
Representation Generator	异步生成 Detailed / Brief summaries	把高延迟 LLM summarization 从 critical path 移走
Attention Scorer	BGE-M3 relevance prediction	本地 dense model，key embedding 缓存，per-step O(M)
Context Builder	relevance + adaptive thresholds -> C _t	真正的 context policy 在这里，但它是算法式 policy，不是主 Agent 权重

这套架构对生产 Deep Research 很重要：Evidence Store 负责“真相保真”，Working Context 负责“本轮计算效率”。两者不应该是同一个对象。

PACE：External Memory + Predictive Attention + Multi-Granularity Context Builder



关键：PACE 不修改主 Agent 权重，也不要求主 Agent 学习额外的“fold”元认知技能

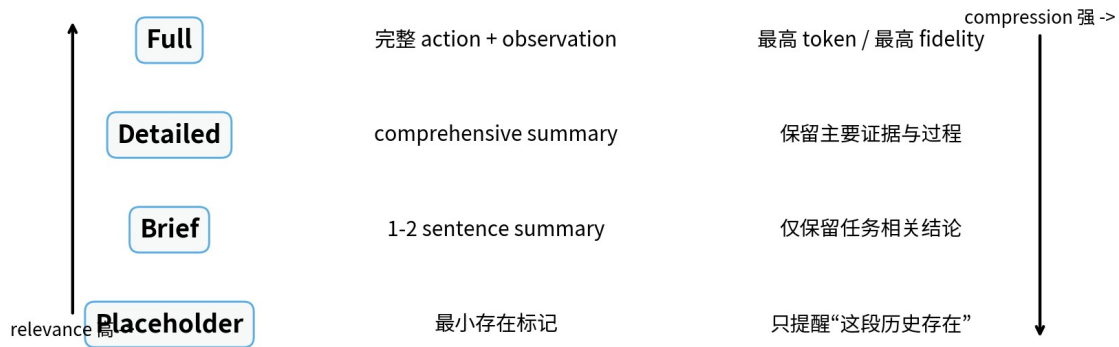
图 4 PACE 的 5-component 系统结构

4. Multi-Granularity Memory：四层历史表示

每个 Chunk_i 保存 id、type、时间信息、四层表示与 key embedding：R_{full} / R_{detailed} / R_{brief} / R_{ph}。Full 是完整内容；Detailed 是 comprehensive summary；Brief 是 1-2 句摘要；Placeholder 只留下极小的存在提示。

为什么四层比“原文/摘要”两层更关键？因为 long-horizon 的相关性不是二元变量。很多历史并不值得占用几千 tokens，但又不能完全消失。Placeholder 的作用不是提供事实，而是保留“可见的记忆索引”；Brief 保留结论；Detailed 保留更丰富证据；Full 留给最相关历史。

同一个历史 Chunk 的四层表示：信息不是“留/删”二元选择



PACE 的核心 know-how：不要只训练“更好的 summary”；先设计多分辨率 memory representation，再让 selection 决定读哪一层。

图5 多粒度 Memory Representation

4.1 可迁移的工程原则

- Evidence Object 应同时支持 Raw、Rich Summary、Compact Summary、Reference Handle。
- Summary 不是 Memory 本体，而是 Memory 的一种 materialized view。
- 表示层级应该由“下一步 utility”选择，而不是由固定 token 阈值选择。
- 最新 observation 需要高保真：PACE 固定保留最近 N=2 个 chunks 的 full text。

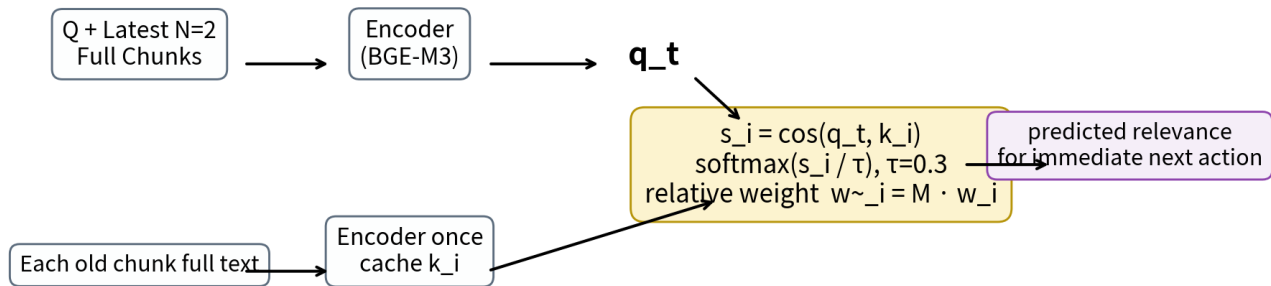
5. Predictive Attention：把历史检索改写成 Next-Step Prediction

PACE 的核心类比来自 Transformer attention：Transformer 为“下一个 token”动态给历史 token 分配权重；PACE 为“下一步 action”动态给历史 interaction chunks 分配权重。

构造 query： $R_t = Q \oplus$ 最近 $N=2$ 个 full chunks； $q_t = \text{Enc}(\text{Truncate}(R_t, L_{\max}))$ 。旧 chunk 的 $k_i = \text{Enc}(\text{Truncate}(R_{\text{full}}^i, L_{\max}))$ 在 chunk 创建时计算并缓存。随后 $s_i = \cos(q_t, k_i)$ 。

论文使用 $\tau=0.3$ 的低温 softmax： $w_i = \exp(s_i/\tau) / \sum \exp(s_j/\tau)$ ，并定义 relative weight $w_{\sim i} = M \cdot w_i$ 。 $w_{\sim i} > 1$ 表示高于平均相关性。这一步的作用是制造“稀疏注意力”：只让少数历史块保留高分辨率。

Context Selection = Next-Step Prediction



为什么低温 softmax 很重要：它把相似度分布“拉尖”，让少数真正相关 chunk 获得高分，其余更容易降级到 Brief / Placeholder。

图 6 PACE 的 Predictive Attention 计算路径

5.1 这里最值得学习的 know-why

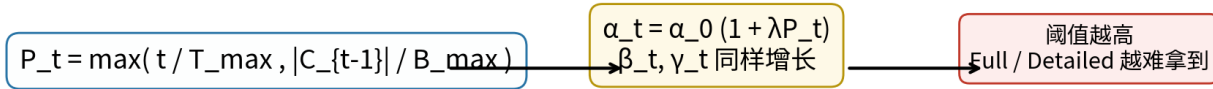
PACE 没有直接问“这个 chunk 对原始问题相关吗？”，而是问“它对下一步 action 相关吗？”。这是非常重要的时间局部性：同一段历史在不同阶段价值不同。一个早期的搜索结果，在探索阶段可能只是 Placeholder；当 Agent 后来发现新的实体关系时，它可能突然成为下一步高相关的 Full chunk。

6. Pressure-Adaptive Compression：预算变成反馈控制信号

仅靠 relevance score 仍不够，因为 20-step task 和 2,000-step task 的 context budget 处境完全不同。PACE 因此定义 compression pressure: $P_t = \max(t/T_{\max}, |C_{t-1}|/B_{\max})$ 。

$T_{\max}$ 取验证集任务长度的 95th percentile, $B_{\max}$ 在主实验中设为 128K。基础阈值 $(\alpha_0, \beta_0, \gamma_0) = (0.4, 0.8, 1.5)$ ，动态阈值按 $\text{threshold}_t = \text{threshold}_0 \cdot (1 + \lambda P_t)$ 增长，默认 $\lambda = 0.5$ 。

Pressure-Adaptive Compression：越接近长程极限，越难保留高分辨率历史



$T_{\max}$ ：验证集任务长度 95th percentile
 $B_{\max}$ ：128K token budget

Default: $(\alpha_0, \beta_0, \gamma_0) = (0.4, 0.8, 1.5)$, $\lambda = 0.5$

这相当于把 Context Budget 变成一个“反馈控制信号”：不是等超限后截断，而是在整个执行过程中逐步提高压缩压力。

图 7 Pressure-Adaptive Threshold 是一个简单但有效的 closed-loop controller

因此 Select(Chunk_i) 是一个四档量化器： $w_i > \gamma_t$ 用 Full； $\beta_t < w_i \leq \gamma_t$ 用 Detailed； $\alpha_t < w_i \leq \beta_t$ 用 Brief；否则 Placeholder。随着 pressure 变大，同一个 relevance score 会被映射到更粗的表示。

6.1 生产级启发

- Context Budget 不应只是 hard limit；应该进入 Context Policy，成为实时状态变量。
- Budget Pressure 至少包含 trajectory progress、token utilization；还可扩展到 latency、money、API quota、KV-cache occupancy。
- 默认阈值应有“舒适区”，避免过度保守导致 overflow，也避免过度激进导致 evidence loss。

7. Glimpse + Async Summaries：为什么 PACE 可以近乎零额外延迟

7.1 Glimpse：压缩错误必须可恢复

PACE 承认 relevance scorer 会犯错，因此提供 $\text{glimpse}(i) \rightarrow R_{\text{full}}^i$ 。Agent 在发现 summary 信息不足时，可主动要求恢复某个历史 chunk 的 full details。每步最多恢复 3 个，以避免 Context 再次失控。

这相当于给“有损 active context”配一个“无损 cold path”。它和 ACM 的 query_memory 有相似精神，但 PACE 的 retrieval handle 是 chunk id，且触发通常是主 Agent 发现信息不足。

7.2 Async Representation Generator

Detailed / Brief summaries 由 Gemini 2.5 Flash-Lite 在后台线程异步生成；如果某个 summary 尚未准备好，Context Builder 根据预算回退到 Full 或 Placeholder，不阻塞主 Agent。

方法	Avg latency / step	相对 ReAct
ReAct	47.63 s	baseline
Summary Agent	56.71 s	+19.1%
PACE	48.08 s	+0.9%

这个结果是 PACE 工程价值的重要证据：Context Management 若进入主 inference critical path，很容易把“节省 token”换成“增加 wall-clock”；PACE 用 cached embedding + async summary 把额外开销压到约 0.45 秒/step。

Source: ACL 2026 paper Appendix B, Table 4.

8. 实验体系与主结果

PACE 在六个 benchmark 上评估：BrowseComp、BrowseComp-ZH、WideSearch、GAIA、xbench-DeepResearch、WebWalkerQA。模型覆盖 WebSailor-32B、Tongyi DeepResearch-30B、DeepSeek-V3.1-671B、Claude-4-Sonnet。所有 context baselines 使用相同 summarization model 与 token budget。

Model	Method	BrowseComp	BC-ZH	WideSearch	GAIA	xbench-DR	WebWalkerQA
WebSailor-32B	ReAct	7.3	20.7	43.2	46.7	58.0	52.3
WebSailor-32B	PACE	13.2	29.3	52.8	59.1	68.0	63.5
DeepSeek-V3.1	ReAct	25.8	44.3	54.8	58.3	66.0	56.4
DeepSeek-V3.1	PACE	35.1	54.8	65.7	69.3	74.0	66.5
Tongyi DR-30B	ReAct	38.2	41.6	52.7	64.6	69.0	66.8
Tongyi DR-30B	PACE	47.6	51.2	64.2	74.0	81.0	78.1
Claude-4-Sonnet	ReAct	11.4	27.2	58.4	65.2	62.0	57.9
Claude-4-Sonnet	PACE	17.8	32.6	67.0	76.4	72.0	65.8

tongyi-deepresearch-30B : PACE 在六项 benchmark 全部最好

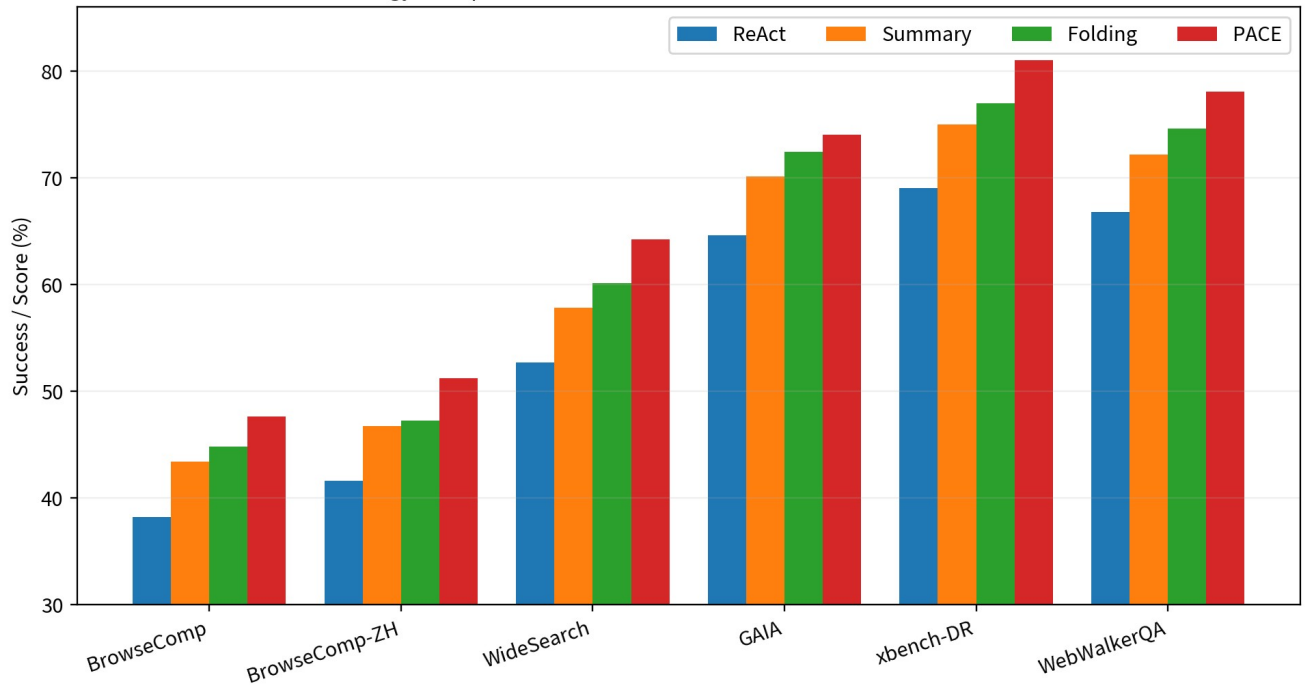


图 8 Tongyi DeepResearch-30B 上的主结果

值得注意：PACE 的收益不是只出现在小模型。Claude-4-Sonnet 在 GAIA 从 Folding Agent 的 70.1 提升到 76.4，说明更强 Agent 也会受到 context bottleneck 限制；改善 read-path 能释放模型已有的 reasoning capability。

9. Ablation：真正重要的组件是什么

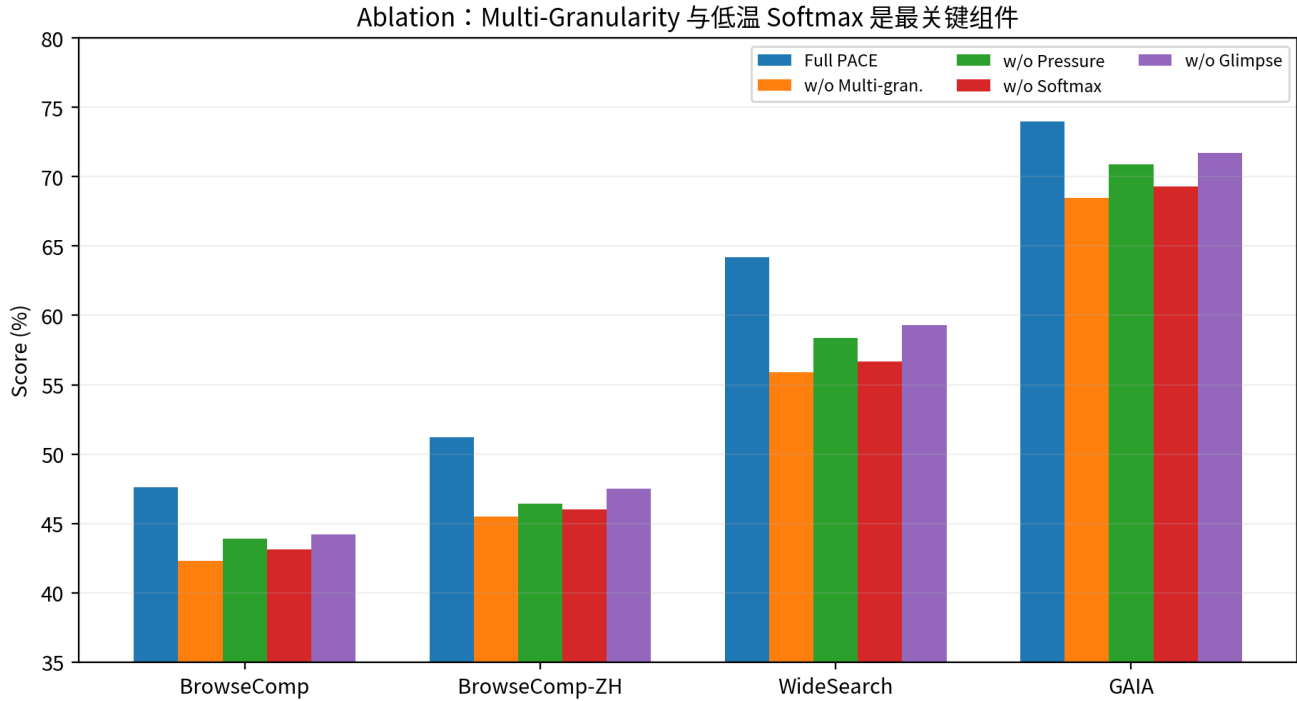


图9 PACE Ablation (Tongyi DeepResearch-30B)

四个消融分别移除：Multi-Granularity、Pressure Adaptation、低温 Softmax、Glimpse。最严重的是移除 Multi-Granularity：四项 benchmark 下降约 5.3-8.3 points；其次是把 τ 从 0.3 恢复到 1.0，下降约 4.7-7.5。

配置	BrowseComp	BC-ZH	WideSearch	GAIA	解释
PACE Full	47.6	51.2	64.2	74.0	完整机制
w/o Multi-gran.	42.3	45.5	55.9	68.5	没有中间表示层，只有 full / placeholder
w/o Pressure	43.9	46.4	58.4	70.9	固定阈值不能随 horizon 调整
w/o Softmax	43.1	46.0	56.7	69.3	相关性分布不够稀疏
w/o Glimpse	44.2	47.5	59.3	71.7	失去错误恢复通路

结论：PACE 不是“检索器 + 摘要器”的简单拼接。真正核心是“多分辨率表示 + 稀疏 relevance + 动态 pressure”三者耦合；Glimpse 是 safety net。

10. Context Growth：从线性膨胀到有界波动

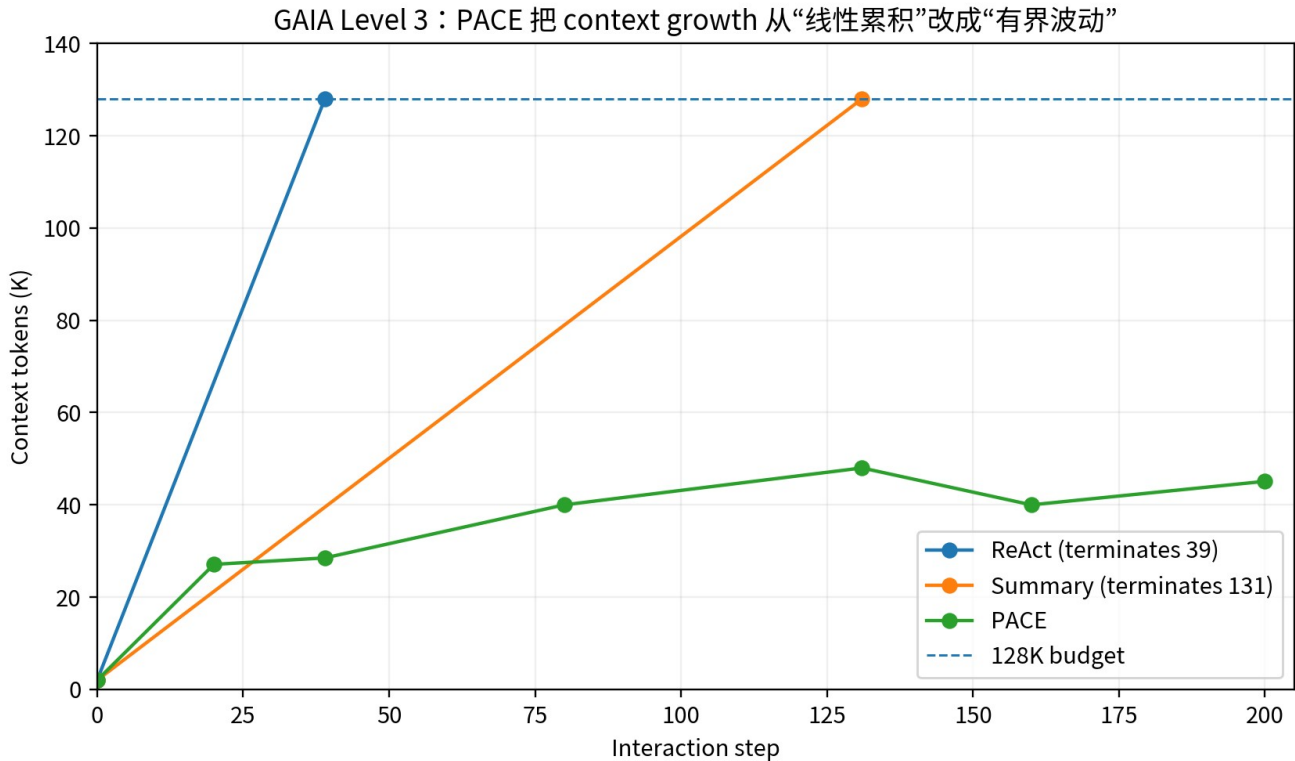


图 10 GAIA Level 3 context growth (依据论文报告关键节点重绘)

ReAct 几乎线性增长，在 128K budget 下 step 39 终止。Summary Agent 将可执行长度延长到 step 131，但其累计摘要仍呈近线性增长。PACE 在前 20 步增长到约 27.1K，随后 pressure-adaptive mechanism 开始主导，context size 在 40-50K 左右波动。

论文给出的关键对比：step 39 时 PACE 28.5K vs ReAct 128K (约 -78%)；step 131 时 PACE 48.0K vs Summary 128K (约 -63%)；step 200 PACE 45.1K，仅比 step 20 的 27.1K 高约 1.7 $\times$ ，但 trajectory 长度已经增长 10 $\times$ 。

这揭示了一个系统级目标：优秀 Context Manager 不一定追求“context 越小越好”，而是追求“context growth 与 horizon 解耦”。最理想的曲线应是 bounded / sublinear，而不是 linear。

11. Task Complexity：任务越难，PACE 越有价值

论文用 GAIA Level 1/2/3 检验 Context Management 与任务复杂度的关系。Level 1 上各方案差距有限；到了 Level 3，Context 的选择质量成为决定性因素。

Model	Method	Level 1	Level 2	Level 3
Tongyi DR-30B	ReAct	83.3	57.6	42.1
Tongyi DR-30B	Folding	85.7	68.2	52.6
Tongyi DR-30B	PACE	88.1	68.2	57.9
Claude-4-Sonnet	ReAct	83.3	59.1	42.1
Claude-4-Sonnet	Folding	85.7	63.6	52.6
Claude-4-Sonnet	PACE	88.1	71.2	63.2

Claude-4-Sonnet Level 3: PACE 63.2，对 ReAct +21.1 points，对 Folding +10.6。这里的 know-why 是：难任务不仅“步数多”，还存在远距离依赖、回溯、跨工具证据组合。静态/不可恢复摘要更容易在早期错误丢失关键细节。

- 短任务：主模型能力占主导，Context Manager 的边际价值较小。
- 中长任务：Context 选择开始成为 capability multiplier。
- 超长任务：Context Policy 直接决定 Agent 是否能继续运行。

12. Ultra-Long Horizon : 4,897 steps 如何得到

标准 benchmark 很少自然产生数千步轨迹，因此作者构造一个 stress test：把 GAIA Level 2 + Level 3 test questions 串成一个连续 meta-task，要求 Gemini 2.5 Pro 在一个 session 内逐题完成。Context window 设为 256K，一旦超过即视为 failure。

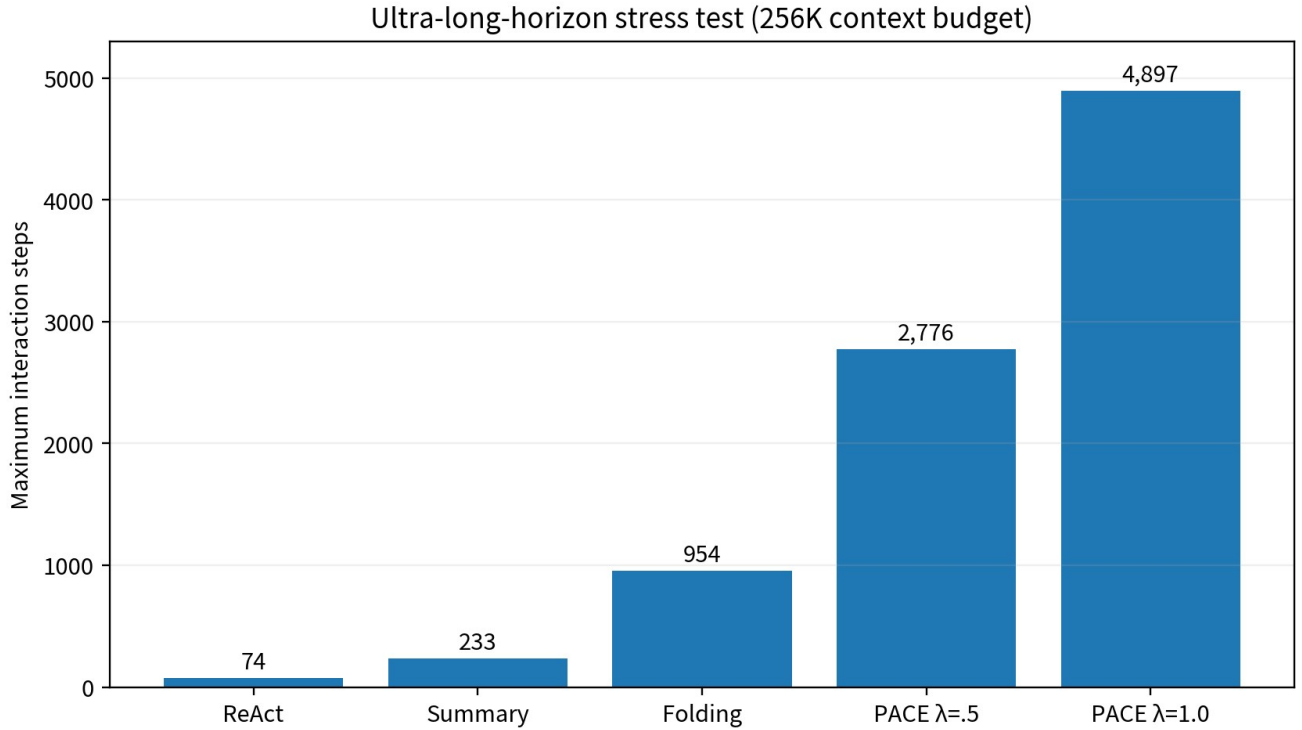


图 11 256K context budget 下的最大可持续 interaction steps

结果：ReAct 74；Summary 233；Folding 954；PACE $\lambda=0.5$ 2,776；PACE $\lambda=1.0$ 4,897。 $\lambda=1.0$ 的极端压缩会损伤一般 benchmark accuracy，但在“不要爆窗口”的 stress-test 目标上，它显著延长寿命。

12.1 这个实验应该怎样正确解读

- 4,897 steps 是 operational longevity，不等于 4,897 个复杂问题全部正确完成。
- Stress test 的 failure 定义是 context overflow，因此它直接测 Context Manager 的可扩展性。
- $\lambda=1.0$ 代表“极端 horizon 优先”配置；主 benchmark 默认 $\lambda=0.5$ ，accuracy 与 longevity 之间存在 trade-off。

13. Hyperparameter 与 Runtime Know-how

Appendix 对 α_0 、 β_0 、 γ_0 、 λ 做了系统分析。默认 (0.4,0.8,1.5,0.5) 在不同模型与 benchmark 上都接近最优，说明这些参数并非只对 Tongyi DeepResearch 过拟合。

λ	BrowseComp	BC-ZH	WideSearch	GAIA	xbench-DR	WebWalkerQA
0.0	42.8	46.1	58.9	70.9	76.0	73.5
0.3	45.5	49.0	62.1	72.4	79.0	76.1
0.5	47.6	51.2	64.2	74.0	81.0	78.1
0.7	46.2	49.5	62.6	73.2	79.0	76.6
1.0	44.3	47.6	60.5	71.7	78.0	74.8

呈明显 inverted-U: $\lambda=0$ 没有 pressure adaptation，会在长程任务中失效； λ 过高又会在后期压得太狠。论文建议一般任务 $\lambda \approx 0.5$ ；极端 >2,000-step 场景可考虑 0.7-1.0。

13.1 Runtime 实现 checklist

- Key embedding 在 chunk creation 时计算并 cache；不要每步重新 encode 全历史。
- Representation Generator 必须异步；summary 未准备好时要有 deterministic fallback。
- 最近 N 个 chunks 固定 full fidelity，PACE 主实验 N=2。
- Glimpse 必须计入下一步 token budget，并设置 per-step cap。
- T_max 用历史分布的 percentile 比固定魔法数字更稳健。
- 监控每一步：active tokens、Full/Detailed/Brief/Placeholder 比例、glimpse rate、embedding latency、summary queue lag。

14. PACE vs ReSum / AgentFold / LongSeeker / AdaCoM / ACM

PACE 的位置：把 Context Management 从“Agent Action”改成“Predictive Memory Read Path”

方法	谁决定	核心操作	可恢复性	是否改主 Agent
ReSum	外部阈值/周期	整体 summary restart	有限	ReSum-GRPO 可训练
AgentFold	主 Agent 每步	multi-scale folding range	有限	SFT 内化 fold skill
LongSeeker	主 Agent 每步	Skip/Compress/Rollback/Snippet/Delete	编辑式	SFT
AdaCoM	外部 manager	rewrite/merge/delete/no-change	取决于实现	Context Manager RL
ACM	主 Agent 主动	manage_context + query_memory	Lossless	OPD/KD
PACE	外部 Context Builder	按预测相关性选择 4 粒度表示	Full history 外存	training-free wrapper

图 12 2025-2026 Context Management design space

从工程视角可以把这些方案分成三类。第一类是“压缩触发器”路线：ReSum 在达到 interval / token pressure 时整体重启。第二类是“Agent Context Action”路线：AgentFold、LongSeeker、ACM 让主 Agent 显式执行 memory/context action。第三类是“外置 Context Policy”路线：AdaCoM 用独立学习 manager，PACE 用非训练式 predictive selector。

系统	核心问题	优点	主要代价/风险
ReSum	何时把 history 总结成 restart state	简单、plug-and-play	summary 一次失真可能不可恢复
AgentFold	主 Agent 应主动折叠哪段 history	多尺度、和 reasoning 强耦合	需要训练 fold skill；主任务与 memory skill 耦合
LongSeeker	Context 是否应成为原子 action space	可表达 rollback/snippet/delete	动作空间复杂，policy 学习负担增加
AdaCoM	能否外置一个 learned Context Manager	冻结主 Agent，适配不同 Agent	额外 manager inference + RL 训练
ACM	主动压缩能否同时 lossless	manage/query memory 可恢复	需要 post-training 主 Agent
PACE	下一步到底需要读哪些历史	training-free、低延迟、可包裹强模型	semantic similarity 不等于真正 causal relevance

如果你的主模型是闭源 API 或不方便 fine-tune，PACE 特别有吸引力；如果你能 post-train 并需要精细回滚/内生 memory action，ACM/LongSeeker 类方案可能更强；如果你希望 Context Policy 与 Agent Policy 完全解耦，则 AdaCoM/PACE 更自然。

15. 生产级 Deep Research 如何复用 PACE

15.1 推荐的六层实现

- ① Raw Trace Store：完整保存 tool calls、observations、documents、citations，不受 active context 生命周期影响。
- ② Evidence Chunker：每次 interaction 形成稳定 ChunkID，并提取 source/provenance 元数据。
- ③ Representation Service：生成 Full pointer、Detailed、Brief、Placeholder，并异步物化。
- ④ Relevance Service：对 Current Research State + Recent Actions 建 query；对旧 chunks 计算 next-step utility。
- ⑤ Budget Controller：根据 token、latency、cost、horizon pressure 动态调节 granularity thresholds。
- ⑥ Recovery Tool：glimpse / query_memory 按 ChunkID 恢复 full evidence，并记录“为什么恢复”。

15.2 比原文再往前一步：Research State 不应只用 Q + 最近两步

PACE 的 query q_t 使用 Original Question + 最近两步 full interaction，简单高效。但生产 Deep Research 可进一步使用结构化 Research State：当前子目标、已覆盖 claims、未解决 gaps、最新 contradictions、citation deficits、budget state。这样 relevance scorer 预测的将不再只是 semantic similarity，而是更接近“对下一步研究决策的 utility”。

15.3 一个可落地的 PACE-MVP

- 每个 tool turn 写入 immutable Chunk(id, raw, source_ids, timestamp)。
- 用小模型异步生成 detailed_summary、brief_summary；placeholder 只保留 id + 8-20 token label。
- 用 embedding 模型缓存 key；每一步用 question + last2 turns + current_subgoal 构造 query。
- softmax τ 从 0.3 起步；使用 4 档阈值；预算超过 60% 后逐步提高 pressure。
- Recent N=2-4 保留 full；最多允许 2-3 个 glimpse。
- Eval 不只看 accuracy：还看 peak active tokens、retrieval recall of critical chunks、glimpse recovery success、latency overhead、summary queue lag。

16. 局限、风险与下一步演进

16.1 论文明确的限制

- Attention Scorer 以 semantic similarity 近似 relevance；它不一定捕获 causal dependency、negation、temporal dependency、工具状态关系。
- Multi-granularity representations 来自固定 summarization model；不是 task-adaptive learned compressor。

这两点决定了 PACE 的下一步很可能是：让 “Context Relevance” 从 embedding similarity 进化成 learned utility / verifier，并让 representation generation 与 retrieval 联合优化。

16.2 我认为最关键的工程风险

- Summary staleness：旧 chunk 的 summary 生成时不知道未来 query，可能遗漏未来才重要的细节；Glimpse 只能在 Agent 意识到缺信息时修复。
- Embedding blind spot：精确数字、代码符号、引用关系、否定/反事实常不适合仅用 dense semantic similarity。
- O(M) scorer：几千步仍可接受，但十万级 trajectory 需要 ANN / hierarchical index / recency partition。
- Placeholder proliferation：几千个 placeholder 本身也会消耗 context，需要进一步聚合或分层索引。
- Context selection 与 evidence correctness 没有直接绑定；Deep Research 仍需要 claim-level verifier / citation graph。

16.3 更完整的下一代公式

Next-Gen Long-Horizon Agent $\approx$ Task Policy $\times$ Predictive Context Read Policy $\times$ Multi-Resolution Memory $\times$ Lossless Evidence Store $\times$ Recovery Path $\times$ Verifier $\times$ Cost/Budget Controller。

17. 结论与资料来源

PACE 最重要的思想不是某个具体阈值，而是将 Context 从“被动累积的历史文本”重新定义为“根据下一步任务动态生成的 memory view”。这让 Context Engineering 更接近数据库 query planning / CPU cache / neural attention：完整状态留在外部，模型每一步只加载最价值的工作集。

- Know-why：长程 Agent 的瓶颈既是 context length，也是 context relevance density。
- Know-how：多分辨率 memory + predictive relevance + pressure feedback + recovery path + async materialization。
- 架构价值：不需要改主 Agent 权重，因此可以包裹 open/closed models；Context Management 成为 runtime primitive。
- 生产原则：Active Context 可以有损，但 Evidence / Trace Store 必须无损；压缩决策必须可观测、可恢复、可评测。

主要资料

1. Lei Wei et al. PACE: Predictive Adaptive Context Extraction for Long-Horizon LLM Agents. ACL 2026, Long Papers, pp. 27184-27199. DOI: 10.18653/v1/2026.acl-long.1252.
2. ACL Anthology official PDF / HTML metadata, accessed 2026-08-13. Paper reports code/data at an anonymized 4open.science snapshot; no verified public GitHub mirror was located during this review, so this report does not claim source-code-level verification beyond the published algorithm/tool descriptions.
3. Related systems used for synthesis: ReSum, AgentFold, LongSeeker/Context-ReAct, AdaCoM, ACM. Comparative statements are architectural synthesis; exact PACE numerical claims are taken from the ACL 2026 paper tables/appendices.